
MuJoCo 学习笔记

Multi-Joint dynamics with Contact

魏舟

2026 年 2 月 21 日

谨以此手册，纪念那段刚刚走进南理工，接触机器人仿真的那段时光。
这个手册根据官方文档和知乎“木一鲸家的程序员”相关文章整理而成。

目录

第 1 章	MuJoCo 下载安装	1
1.1	C/C++ 安装	1
1.2	Python 安装	1
第 2 章	Introduction 简介	3
2.1	Key features 主要特征	3
2.1.1	Generalized coordinates combined with modern contact dynamics 广义坐标系与现代接触动力学的结合	3
2.1.2	Soft, convex and analytically-invertible contact dynamics 软接触、凸接触和解析可逆接触动力学	4
2.1.3	Tendon geometry 肌腱几何结构	4
2.1.4	General actuation model 普适驱动模型	4
2.1.5	Reconfigurable computation pipeline 可重构计算流水线	4
2.1.6	Model compilation 模型编译	4
2.1.7	Separation of model and data 模型与数据的分离	4
2.1.8	Interactive simulation and visualization 交互式仿真与可视化	5
2.1.9	Powerful yet intuitive modeling language 强大而直观的建模语言	5
2.1.10	Automated generation of composite flexible objects 合成柔性对象的自动生成	5
2.2	Model instances 模型实例	5
2.3	Examples 示例	5
2.4	Model elements 模型元素	8
2.4.1	Options	8
2.4.2	Assets 资源	9
2.5	Kinematic tree 运动学树	9
2.5.1	Body 物体	10
2.5.2	Joint 关节	10
2.5.3	DOF 自由度	11
2.5.4	Tree 树	11
2.5.5	Geom 几何	11
2.5.6	Site 站点	12
2.5.7	Camera 摄像机	12
2.5.8	Light 光	12

2.6	Stand-alone	12
2.6.1	Tendon 肌腱	12
2.6.2	Actuator 执行器	13
2.6.3	Sensor 传感器	13
2.6.4	Equality 等式	13
2.6.5	Flex 柔性体	13
2.6.6	Contact pair 接触对	13
2.6.7	Contact exclude	14
2.6.8	Custom numeric 自定义数字	14
2.6.9	Custom text 自定义文本	14
2.6.10	Custom tuple 自定义元组	14
2.6.11	Keyframe 关键帧	14
第 3 章	Computation 计算	15
第 4 章	Modeling 建模	17
4.1	第一性原理	17
4.1.1	第一性原理的内容	17
4.1.2	机器人学	17
4.2	MJCF Mechanisms	18
4.2.1	Kinematics tree	18
4.2.2	Default settings 默认设置	18

MuJoCo 下载安装

§ 1.1 C/C++ 安装

下载路径 <https://github.com/google-deepmind/mujoco/releases> 由于我们是初学者，并不需要开发或修改核心 MuJoCo 代码，所以我们使用预编译的二进制文件即可。

下载并解压缩，可以得到以下目录

bin: 动态链接库、可执行文件等

include: 使用 MuJoCo 所需要的头文件

lib: 使用 MuJoCo 所需要的头文件

model: 模型文件汇总

sample:

simulate:

在 bin 子目录下运行:

```
./simulate ../model/humanoid/humanoid.xml
```

如果能够正常运行，则表示环境没什么问题

我们需要 OpenGL 渲染，所以安装 GLFW 库

```
sudo apt install libglfw3-dev
```

§ 1.2 Python 安装

为了不污染本地的 Python 环境，我们安装 Anaconda，并通过 conda 创建虚拟环境。(如果和我一样污染了也无所谓那就算了，跳过环境配置)

环境配置:

```
conda create --name mujoco-sim python=3.8
```

在 mujoco-sim 环境下:

```
pip install mujoco
```

然后在 model 文件下运行:

```
python -m mujoco.viewer --mjcf=./humanoid/humanoid.xml
```

如果能够正常运行, 则表示环境没什么问题

Introduction 简介

MuJoCo 代表 **Multi-Joint dynamics with Contact**。它是一款通用物理引擎，旨在促进机器人学、生物力学、图形与动画、机器学习以及其他需要快速且精确模拟关节结构与环境交互的领域的研发。最初由 Roboti LLC 开发，2021 年 10 月被 DeepMind 收购并免费提供，并于 2022 年 5 月开源。MuJoCo 代码库可在 GitHub 上的 [google-deepmind/mujoco](https://github.com/google-deepmind/mujoco) 仓库中获取。

MuJoCo 是一个带有 C API 的 C/C++ 库，面向研究人员和开发者。运行时仿真模块经过优化以最大化性能，并运行于由内置 XML 解析器和编译器预先分配的低级数据结构上。用户用原生的 MJCF 场景描述语言定义模型——这是一种旨在尽可能让人类可读和编辑的 XML 文件格式。也可以加载 URDF 模型文件。该库包含带有原生图形界面的交互式可视化，并以 OpenGL 渲染。MuJoCo 进一步揭示了大量用于计算物理相关量的效用函数。

MuJoCo 可用于实现基于模型的计算，如控制综合、状态估计、系统识别、机制设计、通过逆动力学进行数据分析以及机器学习应用中的并行采样。它也可以作为更传统的模拟器使用，包括游戏和互动虚拟环境。

§2.1

Key features 主要特征

MuJoCo 拥有丰富的功能列表。这里我们概述了最著名的几项。

2.1.1 Generalized coordinates combined with modern contact dynamics 广义坐标系与现代接触动力学的结合

物理引擎传统上分为两类。机器人和生物力学引擎在广义坐标或联合坐标系中使用高效且准确的递归算法。但它们要么省略接触动力学，要么依赖早期的弹簧阻尼方法，后者需要非常小的时间步长。游戏引擎采用更现代的方法，通过求解优化问题来求接触力。然而，它们常常采用过度指定的笛卡尔表示法，即在数值上施加联合约束，导致在涉及复杂运动学结构时产生不准确性和不稳定性。MuJoCo 是首个将广义坐标模拟与基于优化的接触动力学两者结合的通用引擎。

2.1.2 Soft, convex and analytically-invertible contact dynamics 软接触、凸接触和解析可逆接触动力学

在现代接触动力学方法中，摩擦接触引起的力或冲量通常被定义为线性或非线性互补问题（LCP 或 NCP）的解，这两种问题都是 NP-hard 的。MuJoCo 基于另一种接触物理的表述，该表述简化为凸优化问题。我们的模型允许软接触及其他约束，并具有唯一定义的逆函数，便于数据分析和控制应用。求解器统一处理摩擦接触，包括扭转摩擦和滚动摩擦、无摩擦接触、关节和肌腱极限、关节和肌腱中的干摩擦，以及多种等式约束。

2.1.3 Tendon geometry 肌腱几何结构

MuJoCo 可以模拟肌腱的三维几何形状——肌腱是遵循包裹和经点约束的最小路径长度弦。该机制类似于 OpenSim，但实现了更为有限的封闭形式包裹选项，以加快计算速度。它还提供机器人专用结构，如滑轮和耦合自由度。肌腱既可用于驱动，也可用于对腱长度施加不等式或等式约束。

2.1.4 General actuation model 普适驱动模型

MuJoCo 通过采用抽象驱动模型实现这一目标，该模型可以具有不同类型的传递、力产生和内部动力学。这些组件可以被实例化，统一建模电机、气动和液压缸、PD 控制器、生物肌肉及许多其他执行器。

2.1.5 Reconfigurable computation pipeline 可重构计算流水线

MuJoCo 有一个顶层步进函数 `mj_step`，可以运行整个前向动力学并推进仿真状态。然而，在许多非仿真应用中，能够运行计算流水线的部分是有益的。为此，MuJoCo 提供了大量可任意组合设置的标志，允许用户根据需要重新配置流水线。此外，许多底层函数也可以直接调用。

2.1.6 Model compilation 模型编译

用户用一种称为 MJCF 的 XML 文件格式定义 MuJoCo 模型。该模型随后由内置编译器编译为低级数据结构 `mjModel`，该结构经过交叉索引并优化以适应运行时计算。

2.1.7 Separation of model and data 模型与数据的分离

MuJoCo 在运行时将仿真参数分为两个数据结构（C 结构体）：

- **mjModel** 包含模型描述，并期望保持不变。它还嵌入了其他结构，包含仿真和可视化选项。
- **mjData** 包含所有动态变量和中间结果。它被用作临时记录，所有函数读取输入并写入输出。它还包含一个预分配且内部管理的栈，因此运行时无需调用内存分配函数。

这种分离使得模拟多个模型以及每个模型的多个状态和控制变得容易，从而促进了多线程采样和有限差分的实现。API 函数格式如下：

```
void mj_step(const mjModel* m, mjData* d);
```

2.1.8 Interactive simulation and visualization 交互式仿真与可视化

原生 3D 可视化工具提供网格、纹理渲染、反射、阴影等功能。该功能用于生成 3D 渲染，帮助用户深入了解物理仿真，包括自动生成的模型骨架、接触位置和法线等视觉辅助。用户可以“进入”模拟，推动物体并观察物理反应，这可用于视觉调试模型或测试控制器。

2.1.9 Powerful yet intuitive modeling language 强大而直观的建模语言

MJCF 的目标是提供 MuJoCo 所有计算能力的访问，同时让用户能够快速开发新模型。得益于其广泛的默认设置机制（类似于 CSS），MJCF 文件通常比许多其他格式更短更易读。

2.1.10 Automated generation of composite flexible objects 合成柔性对象的自动生成

MuJoCo 的软约束可用于建模绳索、布料和可变形的三维物体。建模语言具有高级宏，模型编译器会自动展开为所需的标准模型元素集合。

§ 2.2

Model instances 模型实例

MuJoCo 中有几个被称为“模型”的实体。下表展示了不同介质和描述层级下的实例：

	High level	Low level
File	MJCF/URDF (XML)	MJB (binary)
Memory	mjSpec	mjModel

所有运行时计算均使用 `mjModel`。高层模型（`mjSpec`）存在是为了用户方便，其唯一目的是被编译成低层模型。生成的 `mjModel` 可以保存到二进制文件（MJB），但这些文件是版本特定的且无法反编译。

获得 `mjModel` 的不同路径如下：

- (text editor) → MJCF/URDF file → (MuJoCo parser → mjSpec → compiler) → mjModel
- (user code) → mjSpec → (MuJoCo compiler) → mjModel
- MJB file → (model loader) → mjModel

§ 2.3

Examples 示例

这里有一个 MuJoCo 的 MJCF 格式模型。它定义了一个固定在世界的平面，一个用来更好地照亮物体和投射阴影的灯光，以及一个带有 6 个自由度的漂浮盒子（这就是“自由”关节的作

用)。

```
<mujoco>
<worldbody>
<light diffuse=".5 .5 .5" pos="0 0 3" dir="0 0 -1"/>
<geom type="plane" size="1 1 0.1" rgba=".9 0 0 1"/>
<body pos="0 0 1">
<joint type="free"/>
<geom type="box" size=".1 .2 .3" rgba="0 .9 0 1"/>
</body>
</worldbody>
</mujoco>
```

内置的 OpenGL 可视化工具将该模型渲染为:

如果模拟这个模型, 盒子会掉落到地上。下面给出了无源动力学 (不含渲染) 的基本仿真代码。

```
#include "mujoco.h"
#include "stdio.h"

char error[1000];
mjModel* m;
mjData* d;

int main(void) {
    // load model from file and check for errors
    m = mj_loadXML("hello.xml", NULL, error, 1000);
    if (!m) {
        printf("%s\n", error);
        return 1;
    }

    // make data corresponding to model
    d = mj_makeData(m);

    // run simulation for 10 seconds
    while (d->time < 10)
        mj_step(m, d);

    // free model and data
    mj_deleteData(d);
    mj_deleteModel(m);

    return 0;
}
```

这技术上是 C 文件，但它也是合法的 C++ 文件。事实上，MuJoCo API 兼容 C 和 C++。通常用户代码会用 C++ 编写，因为这增加了便利性，且不会牺牲效率，因为计算瓶颈存在于已经高度优化的模拟器中。

函数 `mj_step` 是顶层函数，将仿真状态推进一个时间步。这个例子当然只是被动动力系统。当用户指定控制或施加力并开始与系统互动时，事情会变得更有趣。

接下来，我们提供一个更详细的例子，说明 MJCF 的若干特征。请考虑以下内容 `example.xml`：

```
<mujoco model="example">
  <default>
    <geom rgba=".8 .6 .4 1"/>
  </default>

  <asset>
    <texture type="skybox" builtin="gradient" rgb1="1 1 1" rgb2=".6 .8 1"
      width="256" height="256"/>
  </asset>

  <worldbody>
    <light pos="0 1 1" dir="0 -1 -1" diffuse="1 1 1"/>
    <body pos="0 0 1">
      <joint type="ball"/>
      <geom type="capsule" size="0.06" fromto="0 0 0 0 0 -.4"/>
      <body pos="0 0 -0.4">
        <joint axis="0 1 0"/>
        <joint axis="1 0 0"/>
        <geom type="capsule" size="0.04" fromto="0 0 0 .3 0 0"/>
        <body pos=".3 0 0">
          <joint axis="0 1 0"/>
          <joint axis="0 0 1"/>
          <geom pos=".1 0 0" size="0.1 0.08 0.02" type="ellipsoid"/>
          <site name="end1" pos="0.2 0 0" size="0.01"/>
        </body>
      </body>
    </body>

    <body pos="0.3 0 0.1">
      <joint type="free"/>
      <geom size="0.07 0.1" type="cylinder"/>
      <site name="end2" pos="0 0 0.1" size="0.01"/>
    </body>
  </worldbody>

  <tendon>
    <spatial limited="true" range="0 0.6" width="0.005">
      <site site="end1"/>
```

```
<site site="end2"/>
</spatial>
</tendon>
</mujoco>
```

该型号是一个 7 自由度臂，“握住”一根绳子，另一端连接一个圆柱体。该字符串为以长度限制的腱（**tendon**）形式实现。肩部有球形关节，肘部和手腕有成对铰链关节。圆柱体内的盒子表示一个自由关节。XML 中的外层 **body** 元素是所需的 **worldbody**。注意，在两个实体之间使用多个关节并不意味着创建虚拟实体。

MJCF 文件包含指定模型所需的最小信息。胶囊由空间中的线段定义——此时只需胶囊的半径。车架的位置和方向是根据其所属的几何推断出来的。惯性性质是在均匀密度假设下从几何形状推断出来的。这两个部位（**site**）之所以被命名，是因为腱的定义需要引用它们，但除此之外没有其他部位的名称。接头轴仅定义于铰链接头，而不定义球接头。碰撞规则是自动定义的。摩擦属性、重力、仿真时间步等都设置为默认值。顶部指定的默认几何颜色适用于所有几何。

除了将编译后的模型保存为二进制 MJB 格式外，我们还可以将其保存为 MJCF 格式或人类可读的文本格式。XML 版本与原始版本类似，而文本版本则包含 **mjModel** 中的所有信息。将文本版本与 XML 版本进行比较，可以发现模型编译器为我们做了多少工作。

§2.4 — Model elements 模型元素

本节简要描述了 MuJoCo 模型中可以包含的所有要素。

2.4.1 Options

每个模型都包含以下三组选项，这些选项始终包含在模型中。如果 XML 文件中未指定这些选项的值，则使用默认值。这些选项的设计允许用户在每个仿真时间步之前更改其值。

mjOption

该结构包含所有影响物理模拟的选项。它用于选择算法并设置其参数，启用和禁用模拟流程的不同部分，以及调整系统级物理属性（如重力）。

mjVisual

此结构包含所有可视化选项。还有其他 OpenGL 渲染选项，但这些选项与会话相关，不属于此模型的一部分。

mjStatistic

该结构包含有关模型的统计信息，这些信息由编译器计算得出：平均体重、模型的空间范围等。包含此结构是为了提供信息，也是因为可视化工具使用它来进行自动缩放。

2.4.2 Assets 资源

Assets 本身并非模型元素。模型元素可以引用 Assets，在这种情况下，Assets 会以某种方式改变引用元素的属性。一个 Asset 可以被多个模型元素引用。由于包含 Asset 的唯一目的就是引用它，而引用只能通过名称进行，因此每个 Asset 都有一个名称。

Mesh 网格

MuJoCo 可以从 OBJ 文件和二进制 STL 文件加载三角网格。虽然任何三角形集合都可以加载并可视为网格，但碰撞检测器仅针对凸包进行工作。编译时提供了缩放网格以及将原始几何形状拟合到网格的选项。网格还可以用于自动推断惯性属性。请注意，网格本身没有颜色，而是使用参考几何体的材质属性来着色。

Skin 蒙皮

蒙皮网格是指其形状可在运行时变形的网格。它们的顶点连接到刚体（骨骼），每个顶点可以属于多个骨骼，从而实现蒙皮的平滑变形。蒙皮纯粹是可视化对象，不影响物理效果。

Height field 高度场

高度场可以从 PNG 文件或自定义二进制格式文件中加载。高度场是一个矩形网格的高程数据。编译器会将数据归一化到 $[0 - 1]$ 范围内。高度场只能被附加到世界对象的几何体引用。

Texture 纹理

纹理可以从 PNG 文件加载，也可以由编译器合成。可视化工具支持两种纹理映射类型：2D 和立方体。2D 映射适用于平面和高度场。立方体映射适用于将纹理“包裹”在 3D 对象周围，或用于创建天空盒。

Material 材质

材质用于控制几何体、地点和肌腱的外观。外观包括纹理映射以及其他与 OpenGL 灯光交互的属性，如：RGBA、镜面反射、光泽度和自发光。

§ 2.5

Kinematic tree 运动学树

MuJoCo 模拟一组刚体的动力学，系统状态以关节坐标表示，刚体被显式地组织成运动学树。树结构由 `mjModel.body_parentid` 给出。顶层“世界”刚体始终存在（id 为 0），并且是其自身的父刚体，即 `body_parentid[0] == 0`，且对于所有其他 i ，满足 `body_parentid[i] < i`。在此顶层静态树之下，可以附加多个运动学树。

运动学回路是不允许的；如果需要环形关节，则应使用等式约束进行建模。

2.5.1 Body 物体

物体具有质量和惯性属性，但不具有任何几何属性。取而代之的是，几何形状（geom）被附加到物体上。每个物体都有两个坐标系：一个用于定义物体本身及确定其他元素相对位置的坐标系，以及一个以物体质心为中心并与其主轴方向一致的惯性坐标系。在每个时间步，MuJoCo 递归地计算正向运动学，从而得到所有物体在全局笛卡尔坐标系中的位置和姿态。物体的数量由 `mjModel.nbody` 给出。

2.5.2 Joint 关节

关节定义在物体内部。它们在物体与其父物体之间创建运动自由度 (DOF)。如果没有关节，物体将与其父物体“焊接”在一起。这与使用过完备笛卡尔坐标的游戏引擎相反，在游戏引擎中，关节会移除自由度而不是增加自由度。

关节有四种类型：

- 球形关节 (Ball joints)
- 滑动关节 (Slide joints)
- 铰链关节 (Hinge joints)
- “自由关节” (Free joints, 用于创建浮动物体)

单个物体可以有多个关节。这样，复合关节就可以自动创建，而无需定义虚拟物体。球形关节和自由关节的方向分量用单位四元数表示，MuJoCo 中的所有计算都遵循四元数的属性。关节的数量由 `mjModel.njnt` 给出。

Joint reference 关节参考姿态

参考姿态是一个存储在 `mjModel.qpos0` 中的关节位置向量。它对应于模型处于初始状态时各关节的数值。

在我们之前的示例中，肘关节被创建为 90° 弯曲状态。但是 MuJoCo 无法识别肘关节，因此默认情况下，它将此关节状态的数值视为 0。我们可以使用关节的 `ref` 属性来覆盖默认行为，并指定初始状态对应于 90° 。所有关节的参考值都会被组合成向量 `mjModel.qpos0`。每当仿真重置时，关节状态都会发生变化：

$$\text{mjData.qpos} = \text{mjModel.qpos0}$$

运行时，关节位置向量是相对于参考姿态进行解释的。具体来说，关节应用的空间变换量为：

$$\text{mjData.qpos} - \text{mjModel.qpos0}$$

此变换是在存储在 `mjModel` 主体元素中的父子平移和旋转偏移量之外的额外变换。

`ref` 属性仅适用于标量关节（滑动关节和铰链关节）。对于球形关节，保存在 `mjModel.qpos0` 中的四元数始终为 $(1, 0, 0, 0)$ ，这对应于零旋转。对于自由关节，浮动体的全局 3D 位置和四元数保存在 `mjModel.qpos0` 中。

Spring reference 弹簧参考姿态

这是所有关节和肌腱弹簧达到其静止长度时的姿态。当关节构型偏离弹簧参考姿态时，会产生弹簧力，且弹簧力与偏离量呈线性关系。弹簧参考姿态保存在 `mjModel.qpos_spring` 中。对于滑动关节和铰链关节，弹簧参考姿态由属性 `springref` 指定。对于球形关节和自由关节，弹簧参考姿态对应于模型的初始构型。

2.5.3 DOF 自由度

自由度与关节密切相关，但并非一一对应，因为球形关节和自由关节都具有多个自由度。可以将关节理解为指定位置信息，而将自由度理解为指定速度和力信息。更正式地说，关节位置是系统构型流形上的坐标，而关节速度是当前位置处该流形切空间上的坐标。自由度具有与速度相关的属性，例如摩擦损失、阻尼和电枢惯性。作用于系统的所有广义力都用自由度空间表示。相比之下，关节具有与位置相关的属性，例如极限和弹簧刚度。自由度并非由用户直接指定，而是由编译器根据关节信息自动生成。自由度的数量由 `mjModel.nv` 指定。

2.5.4 Tree 树

如上所述，运动的物体被组织成运动树。运动树或“树”是一个可移动的物体及其所有后代。因此，世界和其他静态物体都位于全局树结构中，但并不与任何树关联。由于全局树结构采用深度优先组织方式，因此属于同一棵树的所有物体、关节和自由度始终是顺序的。请注意，与物体（如果是静态的）不与任何树关联不同，关节和自由度始终与一棵树关联。`island discovery` 和 `island sleeping` 在树的基础上进行运作

树的数量由 `mjModel.ntree` 给出。

2.5.5 Geom 几何

几何体是刚性连接到物体上的三维形状。多个几何体可以连接到同一个物体上。鉴于 `MuJoCo` 仅支持凸几何体之间的碰撞，而创建非凸物体的唯一方法是将其表示为凸几何体的并集，因此几何体的功能尤为重要。除了碰撞检测和后续的接触力计算之外，几何体还用于渲染，以及在省略物体质量和惯性时自动推断物体的质量和惯性。

`MuJoCo` 支持几种基本几何形状：

- 平面 (Plane)
- 球体 (Sphere)
- 胶囊体 (Capsule)
- 椭球体 (Ellipsoid)
- 圆柱体 (Cylinder)
- 长方体 (Box)

几何体也可以是网格 (Mesh) 或高度场 (Height field); 这可以通过引用相应的资源来实现。几何体具有许多材质属性, 这些属性会影响模拟和可视化效果。几何体的数量由 `mjModel.ngeom` 指定。

2.5.6 Site 站点

站点本质上是轻量级几何体, 代表本体坐标系内感兴趣的位置。站点不参与碰撞检测或惯性属性的自动计算, 但可用于指定其他对象 (例如传感器、肌腱路径和滑块曲柄端点) 的空间属性。站点数量由 `mjModel.nsite` 给出。

2.5.7 Camera 摄像机

模型中可以定义多个摄像机。始终会有一个默认摄像机, 用户可以在交互式可视化工具中使用鼠标自由移动它。然而, 通常情况下, 定义额外的摄像机会更方便, 这些摄像机可以固定在场景中, 也可以附着在某个物体上并随其移动。

除了摄像机的位置和方向之外, 用户还可以调整垂直视场角和瞳距以进行立体渲染, 以及创建立体虚拟环境所需的倾斜投影。在对具有不完美光学元件的真实摄像机进行建模时, 可以分别指定水平和垂直方向的焦距以及非中心主点。摄像机的数量由 `mjModel.ncam` 指定。

2.5.8 Light 光

灯光可以固定在世界物体上, 也可以附加到移动物体上。可视化工具提供对 OpenGL 中完整光照模型 (固定功能) 的访问, 包括环境光、漫反射光和镜面反射光分量、衰减和截止、位置光照和方向光照以及雾效。灯光 (或者更确切地说, 是被灯光照亮的物体) 也可以投射阴影。但是, 与材质反射类似, 每个投射阴影的灯光都会增加一次渲染, 因此应谨慎使用此功能。

§2.6 Stand-alone

在这里, 我们描述不属于单个物体的模型元素, 因此在运动学树之外进行描述。

2.6.1 Tendon 肌腱

肌腱是标量长度单元, 可用于驱动、施加限制和等式约束, 或创建弹簧阻尼器和摩擦损失。肌腱分为两种类型: 固定腱索和空间腱索。固定肌腱是 (标量) 关节位置的线性组合, 适用于模拟机械耦合。空间肌腱定义为穿过一系列指定点 (或过路点) 或环绕指定几何体的最短路径。仅支持球体和圆柱体作为环绕几何体, 且圆柱体在环绕时被视为无限长。为避免肌腱从环绕几何体的一侧突然跳到另一侧, 用户还可以指定首选侧。如果肌腱路径中存在多个环绕几何体, 则必须通过点来分隔它们, 以避免使用迭代求解器。空间肌腱还可以使用滑轮分成多个分支。

2.6.2 Actuator 执行器

MuJoCo 提供了一种灵活的执行器模型，包含三个可独立指定的组件。这三个组件共同决定了执行器的工作原理。通过协调地指定这些组件，可以获得常见的执行器类型。这三个组件分别是传动、激活动力学和力生成。传动组件指定了执行器如何连接到系统的其余部分；可用的类型包括关节、肌腱和滑块曲柄。激活动力学可用于模拟气动或液压缸以及生物肌肉的内部激活状态；使用此类执行器可将整个系统的动力学特性提升至三阶。力生成机制决定了如何将作为执行器输入的标量控制信号映射到标量力，该标量力又如何通过从传动组件推断出的力臂映射到广义力。

2.6.3 Sensor 传感器

MuJoCo 可以生成模拟传感器数据，并将其保存在全局数组 `mjData.sensordata` 中。该结果不会用于任何内部计算；提供该结果是因为用户可能需要它进行自定义计算或数据分析。可用的传感器类型包括触摸传感器、惯性测量单元 (IMU)、力矩传感器、关节和肌腱位置及速度传感器、执行器位置、速度和力传感器、运动捕捉标记位置和四元数以及磁力计。此外，还有一个用户传感器，允许用户代码在传感器数据数组中插入任何其他感兴趣的量。MuJoCo 还具有离屏渲染功能，可以轻松模拟彩色和深度相机传感器。

2.6.4 Equality 等式

等式约束可以在运动学树结构及其定义的关节/自由度之外施加额外的约束。它们可用于创建环形关节，或用于模拟机械耦合。用于强制执行这些约束的内力与其他所有约束力一起计算。可用的等式约束类型包括：

- 将两个物体连接于一点（在运动学树外部创建球形关节）
- 将两个物体焊接在一起
- 固定关节或肌腱的位置
- 通过三次多项式耦合两个关节或两条肌腱的位置
- 将柔性体（即可变形网格）的边约束到其初始长度

2.6.5 Flex 柔性体

柔性体代表可变形网格，可以是 1 维、2 维或 3 维的（因此其元素是胶囊体、三角形或四面体）。与刚性连接到单个实体的静态几何体不同，柔性体的元素是可变形的：它们通过连接多个实体构建，因此实体的位置和方向决定了柔性体元素在运行时的形状。这些可变形元素支持碰撞和接触力，并生成被动和约束力，从而柔和地保持可变形实体的形状。系统提供自动化功能，可以从文件中加载网格，构建与网格顶点对应的实体，并获得相应的可变形网格。

2.6.6 Contact pair 接触对

MuJoCo 中的接触生成是一个复杂的过程。用于检查接触的几何体对可以来自两个来源：自动邻近测试（及其他统称为“动态”的过滤器）以及模型中提供的显式几何体对列表。后者是一

种独立的模型元素类型。由于接触涉及两个几何体的组合，显式指定允许用户以动态机制无法实现的方式定义接触参数。它还有助于微调接触模型。

2.6.7 Contact exclude

这与接触对相反：它指定应从候选接触对生成中排除的实体对（而非几何体）。这对于禁用几何体之间因几何形状而导致不必要的永久接触的实体之间的接触非常有用。

2.6.8 Custom numeric 自定义数字

在 MuJoCo 仿真中，有三种方法可以输入自定义数值：

1. 在 XML 中定义全局数值字段，包含名称和实数值数组。
2. 通过在 XML 元素中设置 `nuser_XXX` 属性，扩展特定元素的自定义数组。
3. 使用 `mjData.userdata`，它不被 MuJoCo 使用，用于存储自定义随时间变化的内容。

2.6.9 Custom text 自定义文本

自定义文本字段可以保存在模型中。它们可用于自定义计算——既可以指定关键字命令，也可以提供其他文本信息。但请勿将它们用于注释。XML 有其自身的注释机制，更适合用于注释。

2.6.10 Custom tuple 自定义元组

自定义元组是 MuJoCo 模型元素的列表，其中可能包含其他元组。它们不被仿真器使用，但可用于指定用户代码所需的元素组。

2.6.11 Keyframe 关键帧

关键帧是仿真状态变量的快照。它包含关节位置向量、关节速度向量、执行器激活值（如有）以及仿真时间。模型可以包含一个关键帧库。关键帧可用于将系统状态重置到感兴趣的点。

Computation 计算

Modeling 建模

在每一个系统的探索中，存在第一性原理，这是一个最基本的命题或假设，不能被省略或删除，也不能被违反。

——亚里士多德

我倾向于从物理框架来研究事物。我们运用第一性原理，而不是类比思维去思考问题，是非常重要的

——埃隆·马斯克

木一鲸家的程序员在他的文章中阐述了第一性原理。我觉得很对啊，第一性原理是物理世界最基本的原理。

所以这一章先记录一下第一性原理。

§4.1 第一性原理

4.1.1 第一性原理的内容

第一性原理的核心思想是将问题分解为最基本、最不可分解的元素或原理。这些原理应当是不依赖于其他原理而能够自洽解释的。通过对这些基本原理的理解，可以构建对复杂系统的深刻理解。

4.1.2 机器人学

机器人最基本的元素是关节 (joint) 与连杆 (link)，这就是机器人的第一性原理。

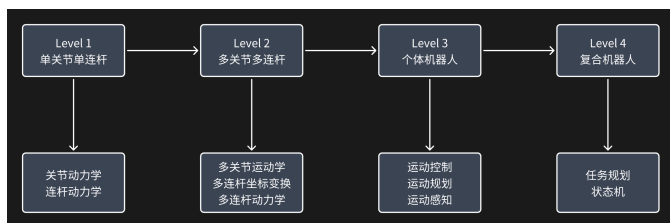


图 4.1: 机器人的第一性原理

level 1: 单关节-单连杆层面 我们通常讨论关节动力学与连杆动力学

level 2: 多关节-多连杆层面 我们通常考虑多关节运动学与连杆坐标变换, 也会讨论多连杆动力学

level 3: 个体机器人层面 我们通常会讨论运动问题, 即移动(足式与轮式)与操作问题。考虑到运动类型, 我们又可以拆分成:

固定基——机械臂

浮动基——足式机器人与轮式机器人

在操作问题中, 我们通常考虑运动控制(关节力控, 笛卡尔空间力控等), 运动规划(轨迹规划, 路径规划等)与运动感知(手眼协调等)

在移动问题中, 我们通常考虑运动控制(车辆动力学与控制等), 运动规划(导航等)与运动感知(SLAM, 三维重建等)

level 4: 复合机器人层面 我们通常会考虑任务规划, 状态机等问题

§4.2 MJCF Mechanisms

MJCF 使用多种模型创建机制, 这些机制涵盖多个模型元素。为避免重复, 本节仅对其进行一次详细描述。这些机制并不涉及计算章节之外的新仿真概念。它们的作用是简化 MJCF 模型的创建, 并允许使用不同的数据格式, 而无需手动转换为标准格式。

4.2.1 Kinematics tree

MJCF 文件的主要部分是由嵌套的 `body` 元素构成的 XML 树。顶层 `body` 元素比较特殊, 称为 `worldbody`。这种树状结构与 URDF 不同, URDF 先创建一系列链接, 然后用连接符将它们连接起来, 连接符指定子链接和父链接。在 MJCF 中, 子 `body` 元素实际上是父 `body` 元素的子元素, 这与 XML 的定义一致。当在实体内部定义关节时, 其功能并非连接父实体和子实体, 而是创建它们之间的运动自由度。如果给定实体内部未定义任何关节, 则该实体与其父实体焊接在一起。在 MJCF 中, 一个实体可以包含多个关节, 因此无需引入虚拟实体来创建复合关节。只需在同一个实体内部定义构成所需复合关节的所有基本关节即可。例如, 可以使用两个滑块和一个铰链来模拟在平面内运动的实体。其他 MJCF 元素可以在由嵌套主体元素创建的树状结构中定义, 特别是 `joint`, `geom`, `site`, `camera`, `light`。当一个元素定义在某个物体内部时, 它会被固定到该物体的局部坐标系中, 并始终随该物体一起运动。引用多个物体或根本不引用任何物体的元素, 则在运动学树之外的单独部分中定义。

4.2.2 Default settings 默认设置

MJCF 拥有完善的默认属性值设置机制。这使得我们能够拥有大量元素和属性, 从而充分展现软件的丰富功能, 同时还能编写简洁易读的模型文件。此外, 该机制还允许用户在一个地方进行更改, 并使其传播到整个模型。我们先来看一个例子。

```

<mujoco>
<default class="main">
<geom rgba="1 0 0 1"/>
<default class="sub">
<geom rgba="0 1 0 1"/>
</default>
</default>

<worldbody>
<geom type="box"/>
<body childclass="sub">
<geom type="ellipsoid"/>
<geom type="sphere" rgba="0 0 1 1"/>
<geom type="cylinder" class="main"/>
</body>
</worldbody>
</mujoco>

```

这个示例实际上无法编译，因为缺少一些必要信息，但我们这里只关注几何图形的 `rgba` 值设置。上面创建四个几何图形，根据默认设置机制，最终将具有以下 `rgba` 值：

表 4.1:

geom type	geom rgba
box	1 0 0 1
ellipsoid	0 1 0 1
sphere	0 0 1 1
cylinder	1 0 0 1

由于没有指定其他类，该立方体使用顶级默认类“**main**”来设置其未定义属性的值。该实体指定了子类“**sub**”，导致该实体的所有子对象（以及它们的子对象等等）都使用类“**sub**”，除非另有指定。因此，椭球体使用类“**sub**”。球体显式定义了 `rgba`，这会覆盖默认设置。圆柱体指定了默认类“**main**”，因此它使用“**main**”而不是“**sub**”，即使在包含该几何体的实体的子类属性中指定了后者。